

Multi-Tenant Provider-Customer Communication System

12-Year Java Full Stack Interview - Conversational Script

Print Settings: Landscape, monospace font (Courier New 10pt), narrow margins

Part 1: The Problem We're Solving

Me: "So here's the situation. You're building a SaaS platform - let's say it's an order fulfillment system, payment processing platform, or document verification service. You have hundreds or thousands of customer companies using your product."

"Each customer has their own systems, and they need to integrate with yours. They send you requests - 'process this order', 'verify this document', 'charge this payment' - and they need to know when it's done."

The Core Challenge

Simple view:

Customer → Your API → Process → Response

Reality:

Customer A → \

Customer B → → Your API → Long-running work → How do they get results?

Customer C → /

(100s of customers, some are spiky, some fail, networks drop)

Me: "The tricky parts are:"

1. **The work takes time** - Maybe 30 seconds, maybe 5 minutes. Can't block HTTP that long.
2. **Networks fail** - Customer sends request, network dies, they retry. Now you might process twice.
3. **Multi-tenant fairness** - Customer A shouldn't kill the system for everyone else.
4. **Reliable delivery** - Once you accept a request, you can't lose it.
5. **Getting results back** - How does the customer know when you're done?

The Pattern We Use:

Step 1: Quick Acceptance

Customer → API → "202 Accepted, here's your ID" (returns in 200ms)

|

v

[Work queued safely]

Step 2: Background Processing

Queue → Worker → Do actual work → Mark complete

Step 3: Notify Customer

Your System → Webhook → Customer's callback URL

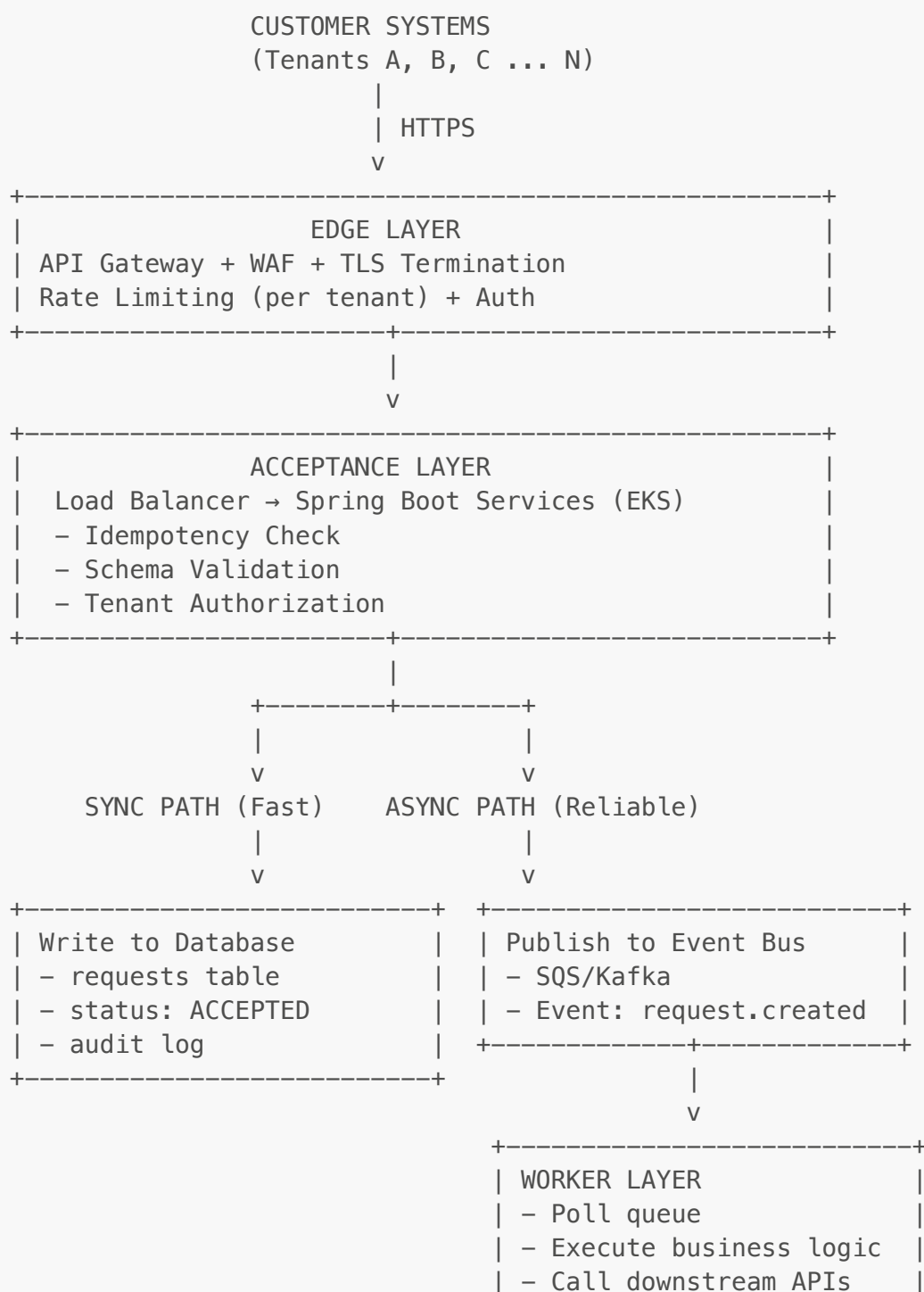
OR

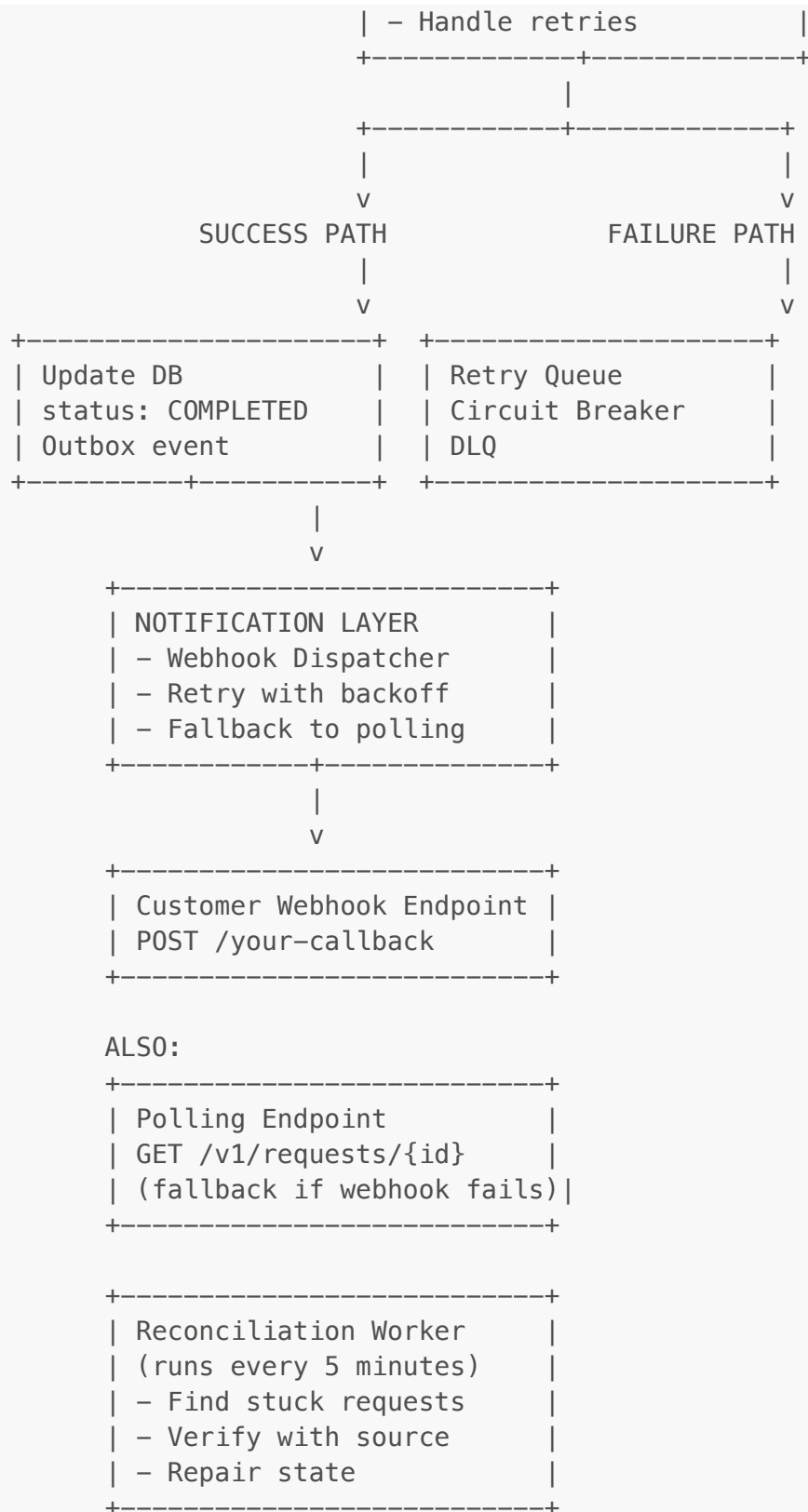
Customer → Poll API → Get status

Part 2: High-Level Architecture

Me: "Here's how I structure this. I split it into two distinct paths - a fast synchronous acceptance path, and a reliable asynchronous completion path."

Full Architecture Diagram





Me: "Let me explain the flow in plain English."

Request Flow (Happy Path)

Timeline of a single request:

t=0ms: Customer POSTs to /v1/requests

```
Headers: { Idempotency-Key: "abc-123", X-Tenant-Id: "customer-a" }

t=50ms: API Gateway validates tenant auth, rate limit OK

t=100ms: Spring Boot receives request
  → Check idempotency table (is abc-123 already processed?)
  → Validate JSON schema
  → Check tenant has permission

t=150ms: Write to database (single @Transactional block):
  → INSERT requests row      (status='ACCEPTED', payload, hash)
  → INSERT idempotency_key   (key, request_id, hash, 24h TTL)
  → INSERT outbox_event      (type='REQUEST_CREATED')
  → INSERT audit_log row     (action='REQUEST_ACCEPTED')
  All 4 inserts commit together – or all rollback

t=180ms: Publish to Kafka/SQS:
  Topic: "requests.created"
  Message: { request_id: 'req-789', tenant_id: 'customer-a' }

t=200ms: Return to customer:
  HTTP 202 Accepted
  {
    "request_id": "req-789",
    "status": "ACCEPTED",
    "message": "Processing started",
    "poll_url": "/v1/requests/req-789"
  }

----- Customer is now unblocked, their API call returned -----

t=2s:   Worker picks up message from queue
  → Load request from database
  → Execute business logic (call payment API, verify document, etc.)
  → This might take 30 seconds

t=30s:  Work completes successfully
  → UPDATE requests: status='COMPLETED', completed_at=now()
  → INSERT outbox_events row (triggers webhook dispatch)

t=31s:  Outbox publisher sees new event
  → Dispatch webhook to customer
  POST https://customer-a.com/webhooks/provider-events
  Body: { request_id: 'req-789', status: 'COMPLETED', result: {...}
}

t=32s:  Customer webhook returns 200 OK
  → Mark webhook attempt as DELIVERED

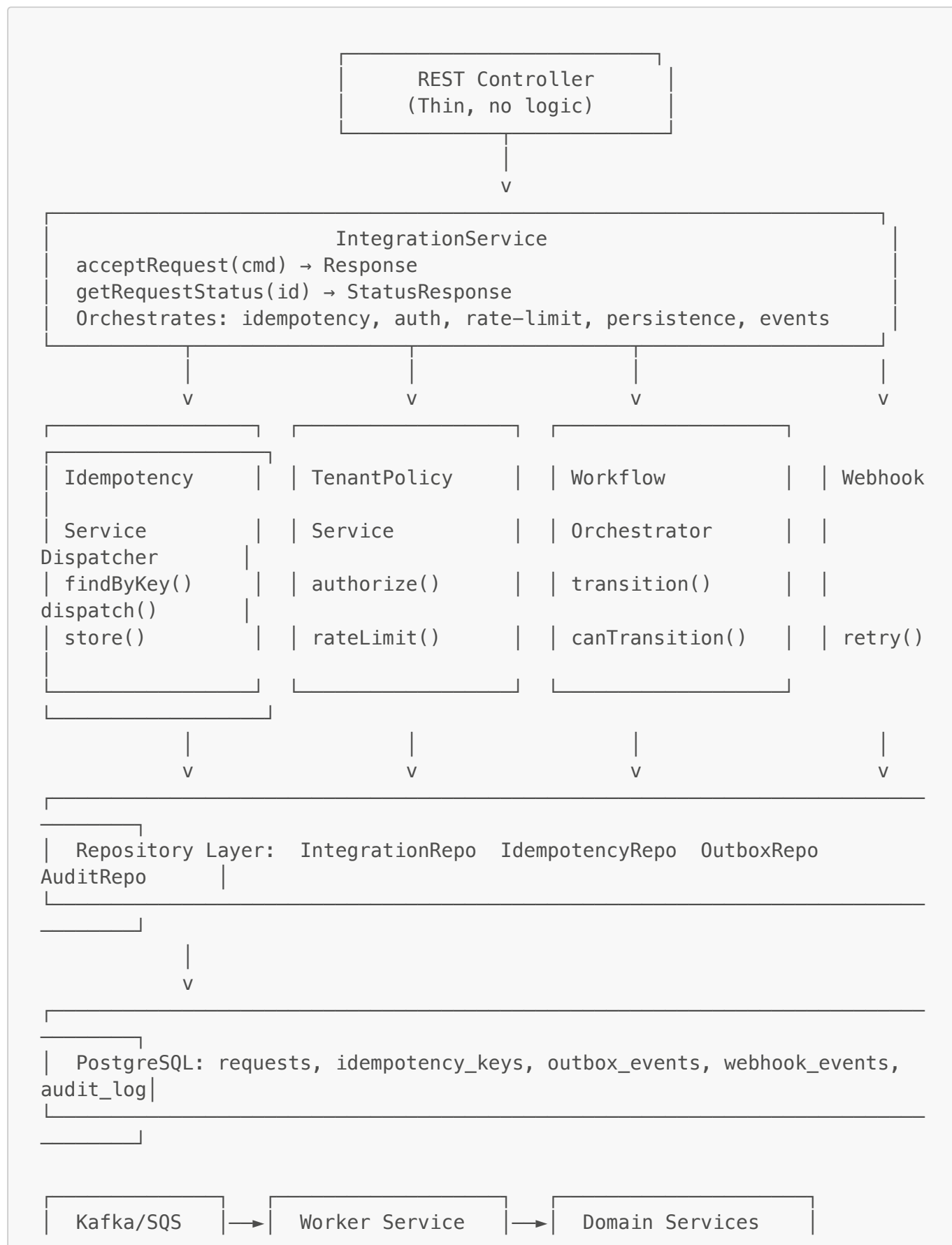
Done! Customer has their result.
```

Me: "Now, that's the happy path. But what makes this production-ready is handling all the failure cases."

Part 3: The Java Implementation - Classes and Responsibilities

Me: "Let me show you how I structure the code. I follow domain-driven design principles with clear separation of concerns."

Class Structure



Event Bus

(Spring @Async)

(Payment, Order)

Key Java Classes

Me: "Let me walk through each class — what it owns, what it must never do, and why those boundaries exist. I've learned through painful experience that blurring these lines is how production incidents start."

Me: "The controller is the thinnest possible layer. I enforce a strict rule: if it isn't about HTTP, it doesn't belong here."

CONTROLLER – THE ONLY THINGS IT IS ALLOWED TO DO

POST /v1/requests

- ✓ Extract headers: Idempotency-Key, X-Tenant-Id
- ✓ @Valid on request body (schema check only)
- ✓ Build AcceptRequestCommand from headers + body
- ✓ Call `integrationService.acceptRequest(command)`
- ✓ Return HTTP 202 Accepted

GET /v1/requests/{requestId}

- ✓ Extract `tenantId` from X-Tenant-Id header
- ✓ Call `service.getRequestStatus(tenantId, requestId)`
- ✓ Return 200 + status, or 404 if not found

NEVER in a controller:

- x Business logic or branching decisions
- x Database calls
- x @Transactional
- x Kafka publishing

WHY THIS DISCIPLINE:

If business logic lives in the controller, you can only test it by starting the full HTTP stack. Keep it thin – test the service.

Me: "The `IntegrationService` is where all the business logic lives. I think of it as a strict 7-step pipeline. The ordering matters — if I swap steps 1 and 3, I burn a rate-limit token on a request we're about to detect as a duplicate. Every ordering decision is intentional."

`IntegrationService.acceptRequest()` – THE 7-STEP PIPELINE
(annotated @Transactional – all DB writes commit as one unit)

STEP 1: CHECK IDEMPOTENCY ← exit early before spending anything

Look up (tenant_id, idempotency_key) in idempotency_keys table

Found + same payload hash → return cached response immediately

Found + different hash → throw 409 Conflict

Not found → continue to step 2

STEP 2: TENANT AUTHORIZATION

Is tenant ACTIVE? Does their plan allow this resource type?

Throws 403 Forbidden if not.

STEP 3: RATE LIMIT CHECK

Token bucket per tenant in Redis.

Throws 429 Too Many Requests if exhausted.

STEP 4: PAYLOAD SCHEMA VALIDATION

Does payload match the agreed API contract for this schema version?

Throws 400 with field-level error detail if not.

STEP 5: FOUR DATABASE INSERTS IN ONE TRANSACTION

① INSERT INTO requests (status='ACCEPTED', payload, hash)

② INSERT INTO idempotency_keys (key, request_id, hash, expires)

③ INSERT INTO audit_log (action='REQUEST_ACCEPTED')

④ INSERT INTO outbox_events (type='REQUEST_CREATED') ← KEY

All 4 commit atomically. If Kafka is down right now, ④ ensures the event will publish when it recovers. Nothing is lost.

STEP 6: (covered by ③ above – audit inside same transaction)

STEP 7: RETURN 202 ACCEPTED

{ request_id, status: 'ACCEPTED', poll_url }

Customer is unblocked. Background processing starts via outbox.

Me: "The worker is a completely separate process — different pod, different JVM, consuming from Kafka. My most important rule about workers: never trust the Kafka message payload for business data. The message is a signal, not the source of truth. Here's the full contract:"

WORKER (RequestWorker) – FOUR NON-NEGOTIABLE RULES

RULE 1: Kafka message = signal only, not data

Message contains only: { request_id, tenant_id }
 Worker loads the full request from PostgreSQL.
 Why? Kafka is not the source of truth. DB is. If the payload in Kafka is from a stale retry, you'd process the wrong version.

RULE 2: Claim with optimistic lock before starting

UPDATE requests SET status='PROCESSING' WHERE status='QUEUED'
 If rowcount=0 → already claimed by another worker → skip
 This is your second duplicate-execution guard.

RULE 3: Retryable vs Non-Retryable failures are different

Retryable (throw → Kafka redelivers with backoff):
 Network timeouts, 503 Service Unavailable, DB connection lost

Non-Retryable (mark FAILED, ACK message, do NOT redeliver):
 400 Bad Request, 403 Forbidden, 404 Not Found from downstream
 Retrying a bad request will keep failing – move on.

RULE 4: Kafka ACK ordering is critical

CORRECT: update DB → commit → ACK Kafka message
 DANGEROUS: ACK Kafka → update DB (pod dies = message lost)

At-least-once delivery is safe because WorkflowOrchestrator
 blocks the illegal PROCESSING → PROCESSING transition.

Me: "The IdempotencyService looks simple but it handles a subtle race condition that most implementations get wrong. Two threads can both check 'has this key been seen?' simultaneously, both get 'no', and both try to INSERT. Here's why that's fine and how the database absorbs the race:"

IDEMPOTENCY SERVICE – THE RACE CONDITION AND WHY IT'S SAFE

findByKey(tenantId, idempotencyKey):

SELECT from idempotency_keys WHERE (tenant_id, key)

Returns null → first time this key has been seen

Returns record → replay scenario:

Same payload hash → return cached response (safe replay)
 Different hash → throw 409 Conflict (key misuse)

store(tenantId, key, requestId, payloadHash):

```
INSERT INTO idempotency_keys (expires_at = now() + 24h)
```

THE RACE – and why the DB constraint wins:

```
Thread 1: SELECT → not found → INSERT ← succeeds
```

```
Thread 2: SELECT → not found → INSERT ← DuplicateKeyException
          PRIMARY KEY constraint fires !!
```

```
Thread 2 catches DuplicateKeyException and re-fetches the record.
```

```
Both threads now have the same request_id.
```

```
No distributed locks needed. The DB is the arbiter.
```

EXPIRY:

```
24-hour TTL on keys. Nightly cleanup job deletes expired rows.
```

```
After 24h, the same idempotency key generates a new request_id.
```

Me: "The WorkflowOrchestrator is a simple class but it does something important: it makes every status transition go through a single auditable gate. Without it, I've seen status updates scattered across the codebase with no consistency."

WORKFLOWORCHESTRATOR – ALLOWED TRANSITIONS AND WHY

```
ACCEPTED    → PROCESSING  (worker picks it up)
ACCEPTED    → CANCELLED   (user cancels before processing)
PROCESSING  → COMPLETED  (success path)
PROCESSING  → FAILED      (permanent failure)
PROCESSING  → UNKNOWN_PENDING_RECON (timeout, unclear outcome)
COMPLETED  → (nothing – terminal)
FAILED      → (nothing – terminal)
```

```
transition(request, newStatus):
```

1. Check `ALLOWED_TRANSITIONS[currentStatus].contains(newStatus)`
2. If not allowed → throw `IllegalStateTransitionException`
(stack trace shows exactly what called it – instant debug)
3. If allowed → UPDATE status, write to `audit_log`

WITHOUT THIS CLASS (what I've seen in codebases):

```
workerService:    request.setStatus(COMPLETED) ← skips audit
reconciler:       request.setStatus(COMPLETED) ← double-counts
webhookService:   request.setStatus(FAILED)     ← wrong caller
```

With the orchestrator, every transition is logged in one place.

Illegal transitions throw immediately with a clear stack trace.

Me: "The WebhookDispatcher has one job: get the result to the customer reliably. I design this defensively because I've been on-call when webhooks fail silently. Every attempt is recorded, every retry is tracked, and there is always a fallback."

WEBHOOKDISPATCHER – DELIVERY ALGORITHM

1. LOAD TENANT WEBHOOK CONFIG (from Redis cache → DB fallback)
webhook_url and webhook_secret for this tenant_id
Cached in Redis (TTL 5 min) – avoids DB hit per dispatch
2. BUILD SIGNED PAYLOAD
Body: { request_id, status, result, event_id, timestamp }
X-Signature: HMAC-SHA256(body, webhook_secret)
Why signature? Customer verifies it's really us, not a replay attack from a third party. OWASP A07 compliance.
3. HTTP POST WITH HARD TIMEOUT (10 seconds, never more)
2xx response → DELIVERED, done
4xx response → PERMANENT FAIL, don't retry (their bug)
5xx / timeout → TRANSIENT FAIL, retry with backoff
4. RETRY SCHEDULE
Attempt 1: immediate
Attempt 2: +1 second
Attempt 3: +5 seconds
Attempt 4: +30 seconds
Attempt 5: +5 minutes
After 5 failures → DEAD LETTER QUEUE + PagerDuty alert
5. CUSTOMER ALWAYS HAS FALLBACK
GET /v1/requests/{id} returns current status at any time.
Customers who can't receive webhooks can poll instead.
6. EVERY ATTEMPT RECORDED
INSERT INTO webhook_attempts (attempt_no, status,
http_status_code, error_message)
When customer says 'I never got it' – show them the log.

Part 4: Database Schema and Indexing Strategy

Me: "Let me show you the database design. This is critical for both performance and correctness."

Database Schema

Me: "Let me draw the table structures instead. I find this more useful in an interview than DDL — you can see at a glance what each column is for and why it exists."

TABLE: requests (main lifecycle, ~10M rows/month)

tenant_id	PK component – every row is scoped to a tenant
request_id	PK component – UUID per request
idempotency_key	UNIQUE per tenant – DB constraint prevents dupes
correlation_id	NULLABLE – customer's own reference ID
status	CHECK: ACCEPTED/PROCESSING/COMPLETED/FAILED
payload	JSONB – the actual request data
payload_hash	SHA-256 of payload – used for idempotency replay
result	JSONB NULLABLE – populated when COMPLETED
error_code	NULLABLE – populated when FAILED
created_at	DEFAULT now()
updated_at	Updated on every status transition
completed_at	NULLABLE – when work actually finished

INDEXES:

ON (tenant_id, status, created_at DESC)

→ "Show me all failed requests for tenant X in the last hour"

ON (status, updated_at) WHERE status IN ('ACCEPTED','PROCESSING')

→ Partial index: only the ~1% of rows that are still active

→ Reconciliation worker uses this to find stuck requests

TABLE: idempotency_keys (fast duplicate lookup, separate)

(tenant_id, idempotency_key)	PK + UNIQUE – DB enforces no dupes
request_id	FK → requests (which request was made)
payload_hash	For replay verification
expires_at	now() + 24h – nightly cleanup target

INDEX ON (expires_at): DELETE WHERE expires_at < NOW() each night

TABLE: outbox_events (reliable event publishing)

```
event_id, tenant_id, request_id, event_type, payload
published_at NULLABLE – NULL means not yet sent to Kafka
```

```
INDEX ON (created_at) WHERE published_at IS NULL
→ Partial index: usually <100 rows in normal operation
→ Outbox publisher queries this every 5 seconds
```

TABLE: webhook_events + webhook_attempts (delivery evidence trail)

```
webhook_events:
  event_id, tenant_id, request_id, payload, signature
  delivered_at NULLABLE – NULL means not yet delivered

webhook_attempts (child of webhook_events):
  attempt_no      INT – which retry this was (1, 2, 3, 4, 5)
  status          DELIVERED / FAILED / TIMEOUT
  http_status_code INT – actual HTTP code received
  error_message   TEXT – what went wrong
```

This is your evidence when a customer says 'I never got it.'
 Show them: attempt 1 at 10:00:00 → 503, attempt 2 → 503,
 attempt 3 at 10:00:06 → 200 DELIVERED.

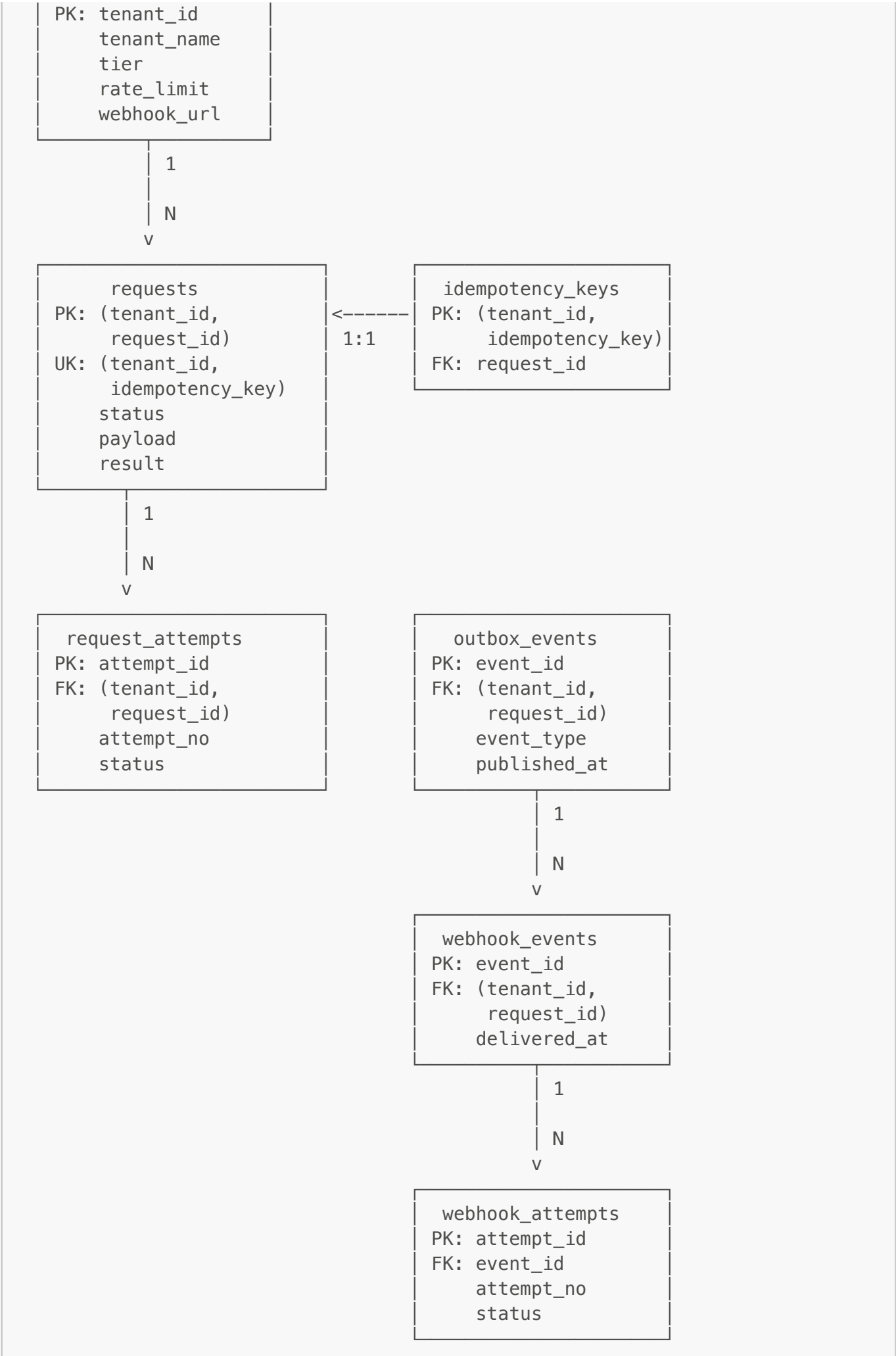
TABLE: dead_letter_queue + audit_log

```
dead_letter_queue: things that need human attention
  tenant_id, request_id, reason_code, reason_detail
  reprocessed_at NULL = ops team hasn't touched it yet
  DLQ size > 0 is always an alert condition.

audit_log: immutable compliance record, NEVER update or delete
  tenant_id, request_id, actor, action, details, created_at
  Every status transition, every manual intervention is here.
  For GDPR erasure: anonymize the details field, not delete the row.
```

ER Diagram

tenants



```

dead_letter_queue
PK: dlq_id
   tenant_id
   request_id
   reason_code
   reprocessed_at

```

```

audit_log
PK: audit_id
   tenant_id
   request_id
   action
   created_at

```

Me: "Notice a few key design choices here:"

1. **Composite primary key** (`tenant_id`, `request_id`) - Natural tenant isolation at database level
2. **Unique constraint on idempotency_key** - Database enforces no duplicates
3. **Separate idempotency_keys table** - Fast lookup without scanning large requests table
4. **Outbox pattern** - Reliably publish events even if Kafka is down
5. **Audit log** - Complete history for compliance and debugging
6. **Attempts tables** - Track retry history for troubleshooting

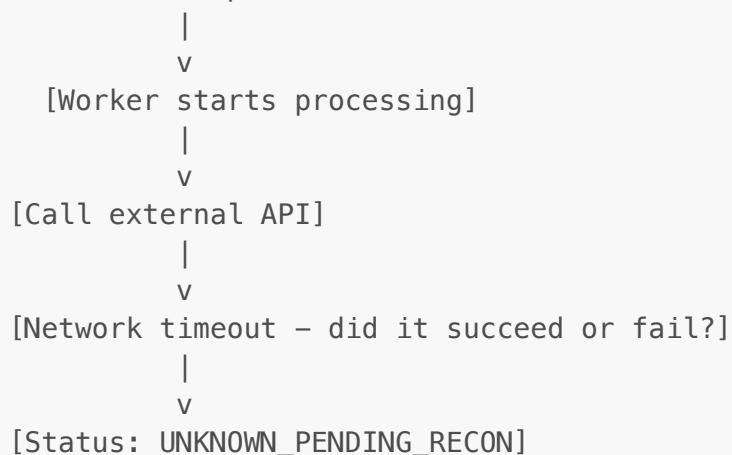
Part 5: Handling Failures - The Critical Part

Me: "This is where 12 years of experience matters. Anyone can build the happy path. Production systems live in failure mode."

Failure Scenario 1: Network Timeout During Processing

Problem:

Customer → POST /v1/requests → 202 Accepted



Solution:

```

Reconciliation Worker
(Runs every 5 minutes)

```

1. Find requests in UNKNOWN state > 5 min old
2. Query source of truth (external API)
3. Compare: was the operation completed?

```
YES → Update status to COMPLETED
NO  → Update status to FAILED
4. Send webhook to customer
```

Me: "The reconciliation worker runs as a Kubernetes CronJob every 5 minutes. It's the safety net for requests that landed in UNKNOWN_PENDING_RECON. Let me walk through exactly what it does:"

RECONCILIATION WORKER – EVERY 5 MINUTES

1. FIND STUCK REQUESTS

```
SELECT * FROM requests
WHERE status = 'UNKNOWN_PENDING_RECON'
      AND updated_at < NOW() - INTERVAL '5 minutes'
```

Only picks up requests that have been stuck for >5 min.
Gives in-flight work time to self-resolve first.

2. QUERY SOURCE OF TRUTH

For each stuck request:
`externalApiClient.queryStatus(request.externalRef)`
The external system is authoritative – whatever it says is truth.

3. REPAIR STATE

External says COMPLETED:
→ `transition(COMPLETED)`, save result, publish webhook

External says FAILED:
→ `transition(FAILED)`, save error, publish failure webhook

External says STILL_PROCESSING:
→ leave in UNKNOWN, pick up again on next 5-minute tick

External unreachable (exception):
→ log error, skip this request, try again next tick

KEY PRINCIPLE:

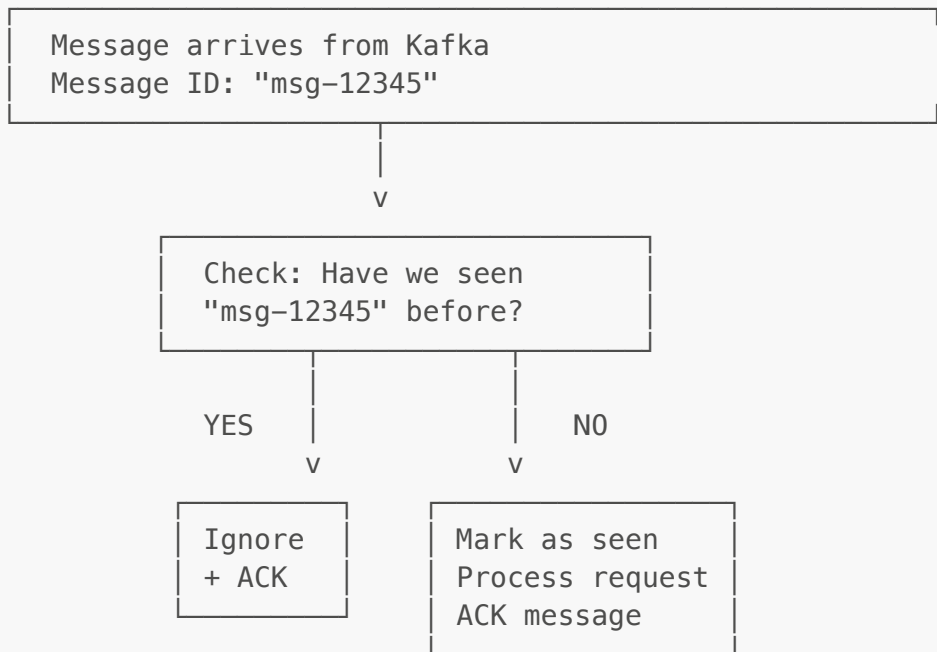
Reconciliation is the ONLY place where UNKNOWN → COMPLETED or
UNKNOWN → FAILED can happen. Centralised and auditable.

Failure Scenario 2: Duplicate Events

Problem:

Worker processes message → Completes successfully → Crashes before ACKing
Kafka redelivers message → Worker processes again → DUPLICATE!

Solution – Deduplication Service:



Me: "Deduplication at the Kafka consumer level is the second guard after idempotency keys. The scenario it covers: worker processes a message, updates the DB successfully, then the pod dies before committing the Kafka offset. Kafka redelivers. Without dedup we process it twice. Here's how Redis SETNX prevents that atomically:"

DEDUPSERVICE – ATOMIC REDIS SETNX PATTERN

```
isDuplicate(messageId):
```

```
Redis SETNX "dedup:msg:{messageId}" = "1" (TTL 24h)
```

```
SETNX succeeded (key was absent) → FIRST TIME → process it
```

```
SETNX failed (key existed) → DUPLICATE → log + discard
```

WHY SETNX?

It's atomic – two concurrent threads racing on the same message ID can only have one succeed. No distributed locks needed.

WHY 24H TTL?

Kafka redelivery windows are typically seconds. 24h gives ample protection without Redis memory growing indefinitely.

Kafka consumer flow:

```
Message arrives with kafka_message_id header
```

```
dedupService.isDuplicate(messageId)?
```

```
YES → log "duplicate detected", return (ACK message)
```

```
NO → proceed to process the request
```


COMBINED WITH WORKFLOWORCHESTRATOR:

Even if dedup misses a duplicate (Redis restart), the orchestrator blocks ACCEPTED → PROCESSING twice. Defence in depth.

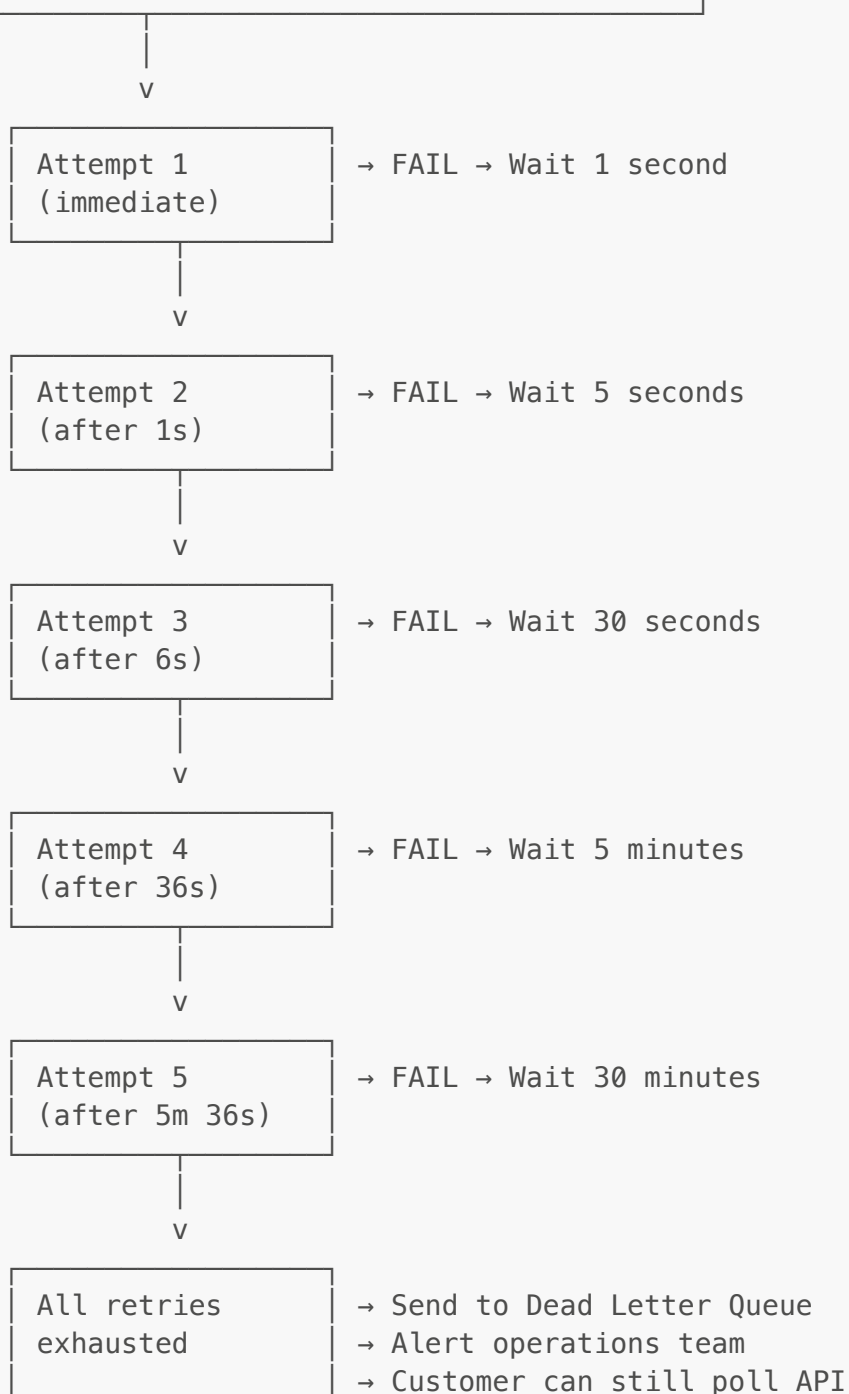
Failure Scenario 3: Webhook Delivery Fails

Problem:

Request completes → Send webhook → Customer server is down (503)

Solution – Retry with Exponential Backoff + DLQ:

Webhook Delivery Flow



Meanwhile, customer has fallback:

Customer Polling (Fallback)

GET /v1/requests/req-789

→ Returns current status even if webhook failed

Failure Scenario 4: Noisy Neighbor (One Tenant Overloads System)

Problem:

Tenant A floods with 1000 requests/sec → Queue backs up → All tenants affected

Solution – Multi-Level Rate Limiting + Fair Queuing:

Level 1: API Gateway Rate Limiting

Tenant A: 100 req/sec

Tenant B: 50 req/sec

Tenant C: 200 req/sec

→ Enforced at edge

v

Level 2: Application-Level Token Bucket

Per-tenant quotas enforced in Spring Boot

→ If exceeded: HTTP 429 Too Many Requests

v

Level 3: Weighted Queue Processing

Worker polls queue with fairness:

– Pull 1 from Tenant A queue

– Pull 1 from Tenant B queue

– Pull 1 from Tenant C queue

– Repeat (round-robin)

Result: No single tenant starves others

TENANT RATE LIMITER – TOKEN BUCKET PER TENANT

```
ConcurrentHashMap<tenantId, RateLimiter>
One RateLimiter instance per tenant, created lazily on first use.
```

```
allowRequest(tenantId):
  Lookup (or create) the tenant's RateLimiter
  Load rate_limit_per_sec from tenants table (cached in Redis)
  RateLimiter.tryAcquire(100ms timeout)
  Returns true → request allowed, decrement token bucket
  Returns false → throttled, return HTTP 429 + Retry-After: 1
```

```
CALLED IN CONTROLLER (before service):
if (!rateLimiter.allowRequest(tenantId)) return 429
```

```
WHY GUAVA RATELIMITER (not Redis)?
For most cases, per-instance token bucket is fast enough.
For strict cross-instance accuracy, use Redis sliding window.
I start with Guava and upgrade to Redis when load demands it.
```

Failure Scenario 5: Database Write Succeeds, Event Publish Fails

Problem:

```
BEGIN TRANSACTION
  INSERT INTO requests ...
COMMIT → Success ✓
```

Publish to Kafka → Network error x

Result: Request exists in DB but worker never processes it!

Solution – Transactional Outbox Pattern:

```
Atomic Write (Same Transaction)

BEGIN TRANSACTION
  INSERT INTO requests (...)
  INSERT INTO outbox_events (...) ← Key!
COMMIT

Either both succeed or both fail
```

↓

```
Separate Outbox Publisher (runs every 5 seconds)

SELECT * FROM outbox_events
WHERE published_at IS NULL
```

```
LIMIT 100
```

```
For each event:
```

```
  Publish to Kafka
```

```
  UPDATE outbox_events
```

```
  SET published_at = NOW()
```

```
→ Eventually consistent, but guaranteed delivery
```

Me: "The outbox pattern solves a classic distributed systems problem: how do you atomically write to a database AND publish to Kafka when they're different systems that can't share a transaction? Let me draw the exact failure scenario it prevents, and then how the outbox fixes it:"

TRANSACTIONAL OUTBOX PATTERN – FULL EXPLANATION

THE PROBLEM (what happens WITHOUT outbox):

```
BEGIN TRANSACTION
```

```
  INSERT INTO requests (status='ACCEPTED') ← succeeds
```

```
COMMIT
```

```
kafkaTemplate.send(...) ← network error!
```

Result: Request exists in DB, Kafka message never sent.
Worker never picks it up. Request stays ACCEPTED forever.

THE FIX – Outbox Pattern:

```
BEGIN TRANSACTION
```

```
  INSERT INTO requests      (status='ACCEPTED') ← succeeds
```

```
  INSERT INTO outbox_events (type='REQUEST_CREATED') ← KEY
```

```
COMMIT ← BOTH or NEITHER, atomically
```

Now the outbox_event is durable in the DB. Can't be lost.

OUTBOX PUBLISHER (separate @Scheduled job, every 5 seconds):

```
SELECT * FROM outbox_events WHERE published_at IS NULL LIMIT 100
```

```
For each event:
```

```
  ① kafkaTemplate.send("requests.created", event.payload)
```

```
  ② UPDATE outbox_events SET published_at=now() WHERE id=?
```

If ① fails (Kafka down):

```
  Log warning, skip, retry on next 5-second tick.
```

```
  Events accumulate in outbox_events. When Kafka recovers,  
  all pending events publish automatically.
```

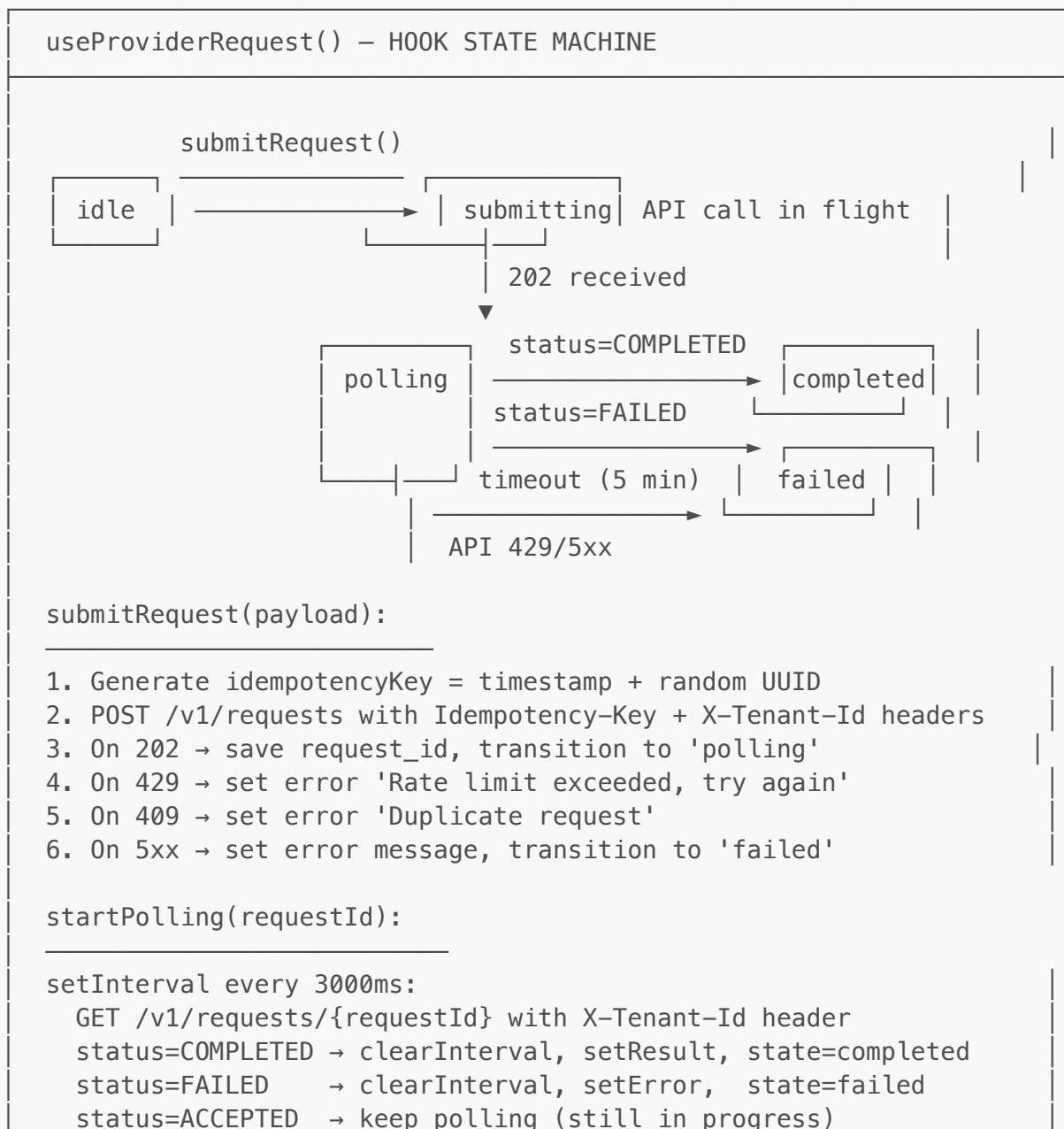
If ① succeeds but ② fails (DB blip):
 Next tick re-fetches the same event (published_at still NULL).
 Kafka gets a duplicate. Worker's DedupService catches it. Safe.

Part 6: React Frontend Integration

Me: "Now let me show you how the frontend interacts with this system."

React Hook for Request Submission

Me: "The React hook is the glue between the asynchronous backend and a synchronous-feeling UI. I encapsulate the entire lifecycle — submit, poll, timeout — in one hook so components stay clean and dumb."



```

network error    → log warning, keep polling (transient)

setTimeout 300000ms (5 min):
  clearInterval, set error 'Request timed out', state=failed

Returns: { submitRequest, status, requestId, result, error }

```

React Component Using the Hook

Me: "The component is pure presentation — it calls the hook and renders the current state. The entire polling and retry logic is invisible to it. Here's what the UI shows in each state:"

OrderSubmissionForm – WHAT EACH STATE RENDERS

```

state: idle
Button: [Submit Order] (enabled)
No banner

state: submitting (API call in flight)
Button: [Submitting...] (DISABLED – prevents double-submit)

state: polling (202 received, waiting for worker)
Button: [Processing...] (DISABLED)
Banner (info): 🔄 Request submitted. ID: req-789
               Processing in background...

state: completed
Banner (success): ✓ Order completed successfully!
                  Confirmation: {result.confirmation_number}

state: failed
Banner (error):   ✗ Order failed: {error message}
Button: [Submit Order] (re-enabled for retry)

KEY UX DECISIONS:
Button disabled during BOTH submitting AND polling.
If user navigates away and back, they can check status via
GET /v1/requests/{requestId} – the poll URL is always available.

```

Webhook Receiver (Customer Side)

Me: "Here's the customer's webhook endpoint — the other side of the integration. I always give customers a reference implementation showing exactly what to do and what NOT to do. The two most common mistakes are: not verifying the signature, and doing heavy processing synchronously before responding."

CUSTOMER WEBHOOK HANDLER – CORRECT IMPLEMENTATION

POST /your-callback (endpoint customer exposes to our platform)

STEP 1: VERIFY SIGNATURE ← ALWAYS DO THIS FIRST

$X\text{-Signature} = \text{HMAC-SHA256}(\text{body}, \text{webhook_secret})$

Recompute on your side and compare. Return 401 if mismatch.

Without this, anyone can POST fake completions to your endpoint and trick your system. OWASP A07 Identification Failure.

STEP 2: RESPOND 200 IMMEDIATELY (before doing any real work)

Return { received: true } immediately.

Our dispatcher has a 10-second timeout. If your handler does heavy DB work synchronously and takes 15s, we assume it failed and retry. You get the same webhook 5 times.

STEP 3: ENQUEUE FOR ASYNC PROCESSING

Push { request_id, status, result } to your own internal queue. A separate worker processes it. Keeps your webhook fast.

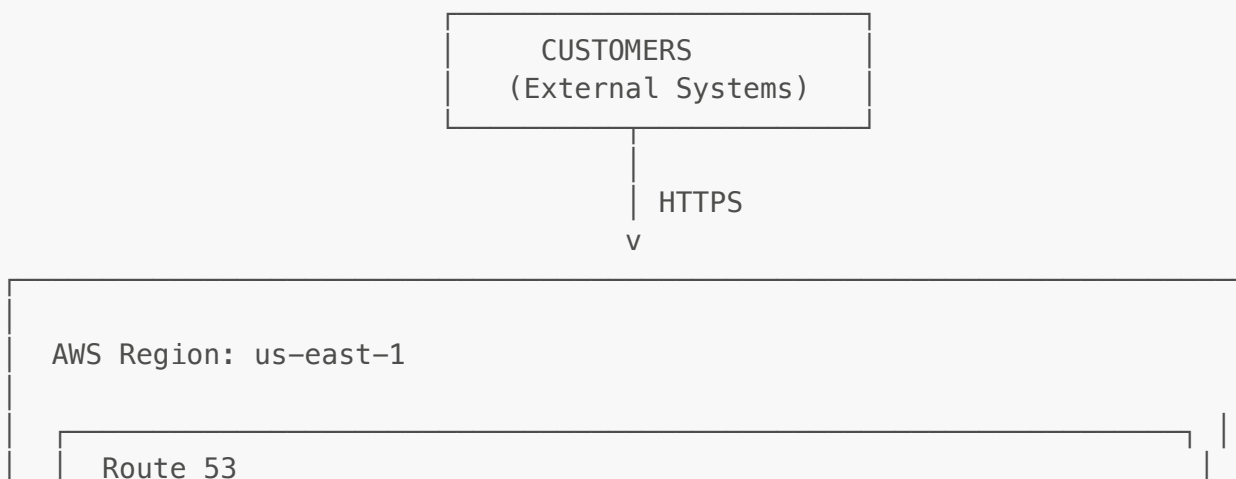
STEP 4: IF YOUR HANDLER RETURNS 5xx

Our dispatcher retries up to 5 times with backoff.

Make your handler idempotent – receiving the same webhook twice must be safe. Check if request_id was already processed.

Part 7: AWS Deployment Architecture

Me: "Let me show you how this deploys on AWS."



DNS: api.yourplatform.com

v

CloudFront (Optional CDN)

- DDoS protection
- Geo-blocking if needed

v

API Gateway (Regional)

- Authentication (Cognito/Custom authorizer)
- Rate limiting (per tenant)
- Request validation
- WAF (Web Application Firewall)

v

Application Load Balancer (ALB)

- Multi-AZ
- Health checks
- SSL termination

v

EKS Cluster (Kubernetes)

API Service Pods (3 replicas)

- Spring Boot
- CPU: 2 cores, RAM: 4GB
- Auto-scaling: 3-10 pods based on CPU/requests

Worker Service Pods (5 replicas)

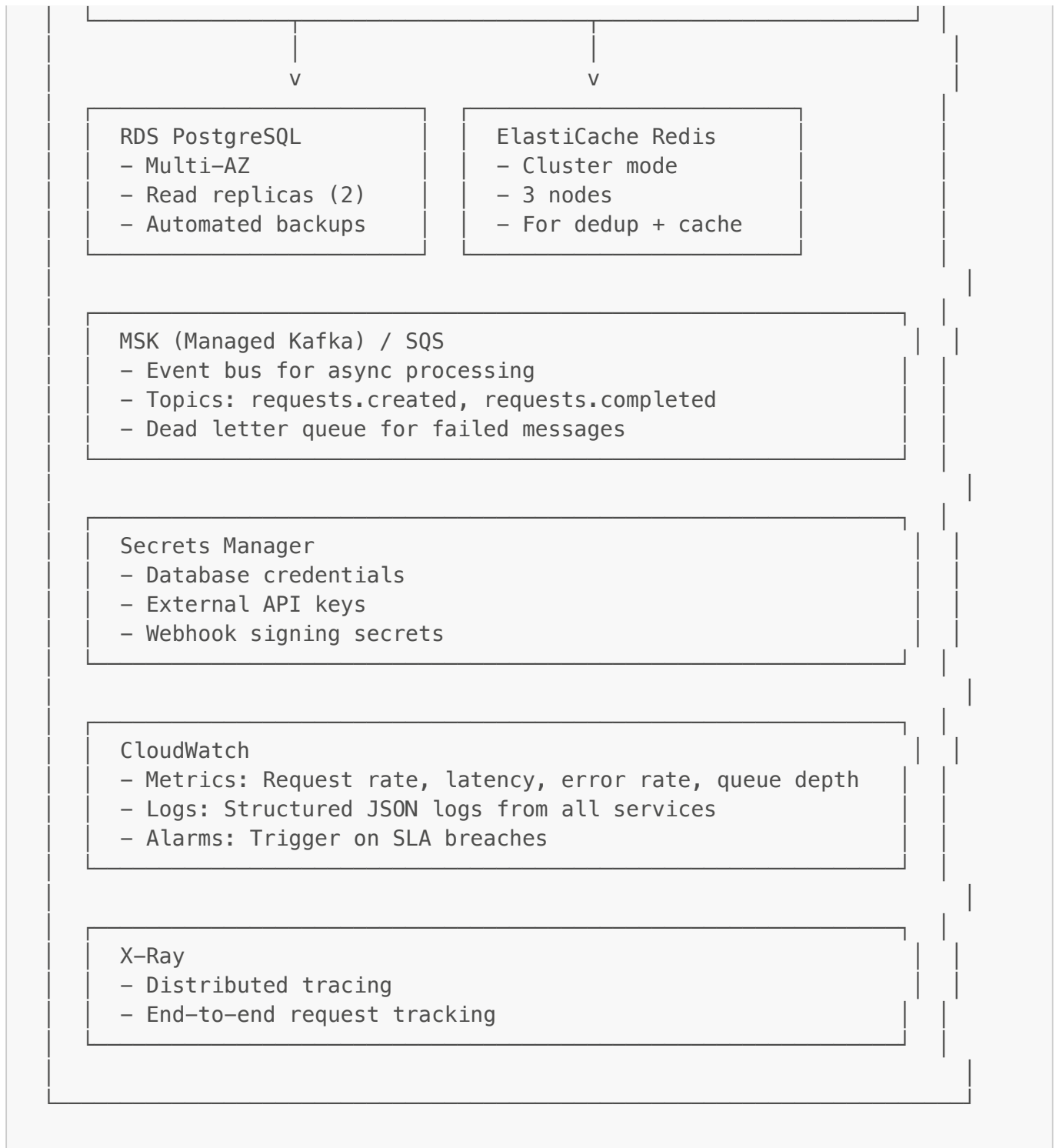
- Process async requests
- CPU: 1 core, RAM: 2GB
- Auto-scaling: 5-20 pods based on queue depth

Webhook Dispatcher Pods (2 replicas)

- Send callbacks to customers

Reconciliation Worker (1 pod)

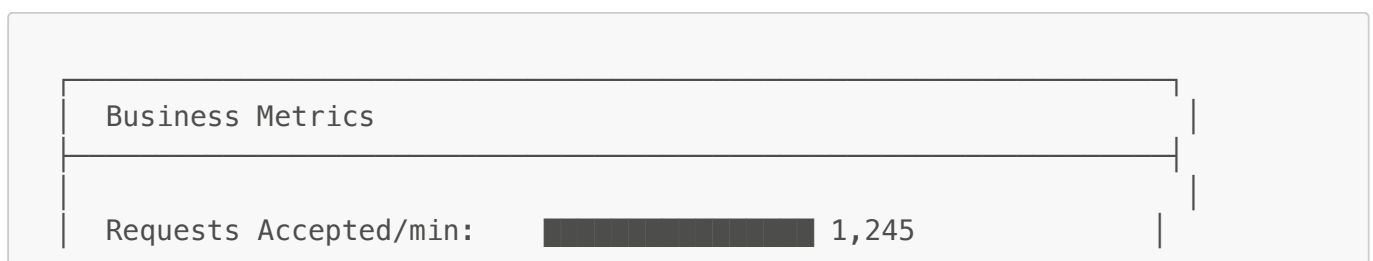
- CronJob runs every 5 minutes



Part 8: Observability and Operations

Me: "Production systems need comprehensive observability. Here's what I monitor."

Key Metrics Dashboard



Completion Success Rate:

98.7%

P95 Completion Latency:

8.2s

Webhook Delivery Rate:

97.3%

Technical Metrics

API P95 Latency:

187ms

Worker Queue Depth:

234 msgs

Queue Lag:

3.2s

Retry Rate:

4.2%

DLQ Size:

12 msgs

Unknown Pending:

5 requests

Per-Tenant Metrics

Tenant A:

524 req/min (99.2% success)

Tenant B:

189 req/min (98.1% success)

Tenant C:

782 req/min (97.8% success)

Tenant D:

98 req/min (99.5% success)

Throttled: Tenant C (23 requests in last minute)

Critical Alerts

Alert Name	Threshold	Action
Completion success < 95%	5 min	Page on-call
Queue lag > 60 seconds	3 min	Auto-scale workers
Webhook failure rate > 10%	10 min	Circuit breaker
Unknown pending > 100	15 min	Increase recon freq
DLQ size > 1000	Immediate	Page ops team
API P95 latency > 500ms	5 min	Scale API pods
Database CPU > 80%	3 min	Alert DBA

Distributed Tracing Example

Request trace: correlation_id=abc-123

Span 1: API Gateway → ALB → EKS	[42ms]
└─ Auth check	[8ms]
└─ Rate limit check	[3ms]
Span 2: IntegrationService.acceptRequest	[145ms]
└─ Idempotency check (Redis)	[12ms]
└─ Tenant policy check	[5ms]
└─ Schema validation	[18ms]
└─ Database write (requests table)	[87ms] ▲
└─ Outbox write	[15ms]
└─ Publish to Kafka	[8ms]
Span 3: Worker.processRequest	[8,245ms]
└─ Load request from DB	[23ms]
└─ Execute business logic	[8,187ms] ▲
└─ Call external payment API	[7,892ms] ▲▲
└─ Process response	[234ms]
└─ Validate	[61ms]
└─ Update request status	[28ms]
└─ Publish completion event	[7ms]
Span 4: WebhookDispatcher.dispatch	[1,234ms]
└─ Load tenant config	[12ms]
└─ Sign payload	[3ms]
└─ HTTP POST to customer webhook	[1,189ms] ▲
└─ Record attempt	[30ms]

Total: 9,666ms (most time in external API call – expected)

Me: "Notice how X-Ray shows exactly where time is spent. The external payment API took 7.9 seconds – that's expected, but if it suddenly jumps to 30 seconds, I'd know there's an issue with the payment provider, not our system."

30-Second Elevator Pitch

Me: "Provider-customer communication in a multi-tenant SaaS is fundamentally about managing asynchrony and failure. I decouple request acceptance from completion using an event-driven architecture with idempotency, state machines, and reliable delivery. I enforce tenant isolation at multiple levels — rate limits, quotas, resource partitioning. I design for observability — structured logs, distributed tracing, per-tenant metrics. And I build recovery mechanisms — reconciliation, retries with backoff, dead letter queues. The result is a system that scales to thousands of tenants, handles millions of requests, and degrades gracefully under failure."

Quick Reference Card (Print This!)

MULTI-TENANT PROVIDER-CUSTOMER SYSTEM – CHEAT SHEET

FLOW:

1. Accept fast (202) → 2. Queue work → 3. Process async →
4. Notify via webhook → 5. Customer polls as fallback

KEY PATTERNS:

- Idempotency: Unique key per request
- Outbox: Reliable event publishing
- State Machine: Legal transitions only
- Dedupe: Ignore duplicate events
- Reconciliation: Repair stuck requests
- Circuit Breaker: Isolate tenant failures

KEY TABLES:

- requests: Main lifecycle
- idempotency_keys: Prevent duplicates
- request_attempts: Retry history
- outbox_events: Reliable publishing
- webhook_events: Delivery tracking
- dead_letter_queue: Manual intervention
- audit_log: Complete history

KEY METRICS:

- Completion success rate
- P95 completion latency
- Webhook delivery rate
- Queue lag
- Unknown pending count
- Per-tenant throttle rate

FAILURE MODES:

- Network timeout → Reconciliation finds truth
- Duplicate events → Dedupe in Redis
- Webhook fails → Retry + DLQ + polling
- Noisy tenant → Rate limit + queue partition
- DB write succeeds, publish fails → Outbox pattern

TECH STACK:

- Java 17 + Spring Boot
- PostgreSQL (Multi-AZ)
- Redis (dedupe + cache)
- Kafka/SQS (event bus)
- EKS (container orchestration)
- CloudWatch + X-Ray (observability)

Part 11: Principal-Level Deep Dives (15-Year Gaps)

11.1 Data Isolation Strategy — The Most Commonly Asked Question

Interviewer: "You mentioned multi-tenant. How do you actually isolate one tenant's data from another?"

Me: "This is the foundational architectural decision for any multi-tenant system, and there's no single right answer — it's a trade-off between isolation, cost, and operational complexity. Let me walk through all four options and tell you which I'd choose and why."

DATA ISOLATION STRATEGIES – TRADE-OFF MATRIX				
Strategy	Isolation	Cost	Ops burden	Best for
A: Shared schema	Lowest	Lowest	Low	High
tenant_id col	SQL bug leaks	1 DB cluster		tenant
	everything			count
B: Schema-per-tenant	Medium	Medium	Medium	10s-100s
search_path	Connection	1 DB, N	migrations	of
	misconfigured	schemas	× N	tenants
	leaks data		schemas	
C: DB-per-tenant	Highest	Highest	High	<10
enterprise	Full kernel	N DBs	migrations	
	isolation		× N DBs	tenants
D: Hybrid (tiers)	Configurable	Balanced	Medium	SaaS
free=shared	Free tenants	1 shared +		with
paid=dedicated	share risk	N enterprise		tiers

Me: "For the system I described, I'd use **Option A (shared schema) with Option D for enterprise tiers**. Here's my reasoning:

We might have thousands of tenants. A DB per tenant at that scale means thousands of connection pools, thousands of migration runs, thousands of backup schedules. That's not operationally viable.

Shared schema with `tenant_id` on every table is simple and scales. But it has one critical weakness — a SQL bug that omits the `WHERE tenant_id=?` filter leaks everyone's data. So I harden it with two layers:"

HARDENING SHARED SCHEMA AGAINST DATA LEAKAGE

Layer 1: PostgreSQL Row-Level Security (RLS)

```
ALTER TABLE requests ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON requests
  USING (tenant_id = current_setting('app.tenant_id'));

SET app.tenant_id = 'customer-a';
SELECT * FROM requests;  -- only sees customer-a rows
                        -- even if WHERE clause is missing !!
```

Layer 2: Application-Level Guard in Spring Boot

```
@Aspect
public class TenantIsolationAspect {
    @Before("execution(* *Repository.*(..))")
    public void assertTenantContext() {
        if (TenantContext.get() == null)
            throw new MissingTenantContextException();
    }
}
```

Layer 3: Integration Tests That Prove Isolation

```
@Test
void tenantA_cannotSee_tenantB_data() {
    createRequest(TENANT_A, "req-A");
    createRequest(TENANT_B, "req-B");

    TenantContext.set(TENANT_A);
    var results = requestRepo.findAll();
```

```
assertThat(results).allMatch(r -> r.getTenantId() == TENANT_A);  
assertThat(results).noneMatch(r -> r.getTenantId() == TENANT_B);  
}
```

"For enterprise tenants with data sovereignty requirements, I'd promote them to a dedicated schema or dedicated database. This is the hybrid tier approach — same application code, different storage backend per tier."

11.2 Zero-Downtime Schema Migrations at Scale

Interviewer: "How do you deploy database changes without downtime when you have 10 million rows shared across thousands of tenants?"

Me: "This is where most teams get burned. Let me show you the wrong way first, and then the right way."

The wrong way is what developers do instinctively:"

NAIVE MIGRATION (causes downtime at scale)

```
ALTER TABLE requests ADD COLUMN priority INT NOT NULL DEFAULT 5;
```

On 10M rows → PostgreSQL rewrites the entire table

Duration: minutes

Impact: Table locked for reads AND writes during rewrite

Result: All API calls fail with lock timeout

Me: "The correct approach is the **expand-contract pattern**. You never change the schema in one step — you expand first (add new column, backwards compatible), deploy the new code that uses both old and new, then contract (remove old column once all code is off it)."

EXPAND-CONTRACT PATTERN — Zero Downtime

PHASE 1: EXPAND (backwards compatible)

```
ALTER TABLE requests
```

```
  ADD COLUMN priority_v2 INT;  -- nullable, no default
```

→ Safe: no table rewrite, no lock

→ Old code still works (ignores the new column)

PHASE 2: DEPLOY CODE that writes to BOTH columns

```
INSERT INTO requests (priority, priority_v2, ...)
VALUES (5, 5, ...) -- write to both during migration window

PHASE 3: BACKFILL old rows

UPDATE requests
SET priority_v2 = priority
WHERE priority_v2 IS NULL
LIMIT 10000; -- run in batches, never lock full table

(run as background job, 10K rows at a time, sleep between batches)

PHASE 4: DEPLOY CODE that reads from priority_v2 only

PHASE 5: CONTRACT (remove old column)

ALTER TABLE requests DROP COLUMN priority;
ALTER TABLE requests RENAME COLUMN priority_v2 TO priority;

→ Total migration: 0 seconds of downtime
→ Each phase independently deployable and rollback-safe
```

Me: "In Spring Boot, I use **Flyway** with a strict naming convention: `V{version}__description.sql`. For high-risk migrations, I also use `pg_repack` which rewrites the table in the background without holding locks. The key principle is: if your migration can't run in under 1 second, it needs to be broken into phases."

11.3 Saga Pattern — Distributed Transactions Across Services

Interviewer: "What if processing a request involves multiple steps across different services — and one of them fails halfway through?"

Me: "This is the distributed transactions problem, and it comes up constantly in microservices. Let me give you a concrete example from our system:

Processing an order means:

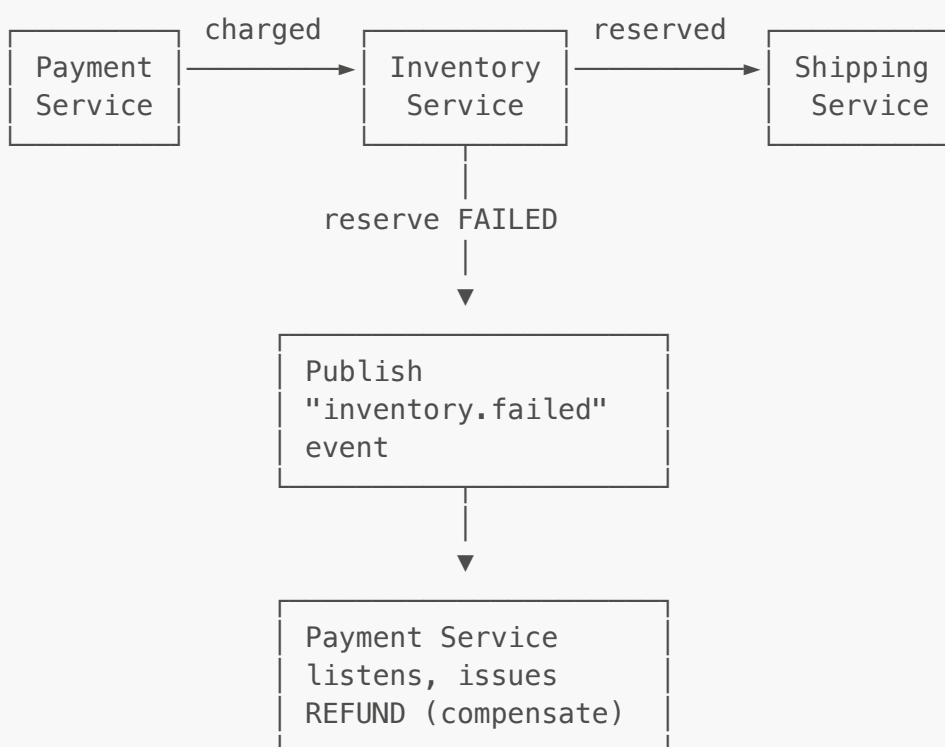
1. Charge the customer's card (Payment Service)
2. Reserve the items (Inventory Service)
3. Notify the warehouse (Shipping Service)

If step 2 fails after step 1 already charged the card — we have a mess. The customer was charged for something they can't receive.

We can't use a database transaction across services — they're in different databases. The solution is the **Saga pattern**."

SAGA PATTERN – Choreography vs Orchestration

OPTION A: CHOREOGRAPHY (Event-Driven)



Pro: No central coordinator, fully decoupled

Con: Hard to follow, hard to debug, "saga" logic scattered

OPTION B: ORCHESTRATION (Recommended for complex flows)

Order Saga Orchestrator (in our IntegrationService)

Step 1: chargeCard() → SUCCESS → continue

Step 2: reserveInventory() → FAIL

Compensation sequence (in reverse):

Step 1 rollback: refundCard() → SUCCESS

Saga ends: FAILED (with clean state)

Pro: Single place to read the entire workflow

Con: Orchestrator becomes a bottleneck if not designed carefully

Me: "In practice I use a SagaBuilder that encodes this exactly. Each step is registered with a forward function AND a compensating function. The builder executes steps in order and runs compensations in reverse on failure. The critical discipline is that every compensating action must be idempotent — calling refundCard() twice must produce the same result as calling it once, because compensations can fail and be retried."

SAGA ORCHESTRATOR – HOW TO BUILD ONE

```
SagaBuilder.create()
  .step("charge-card",
        forward:    paymentService.charge(order),
        compensate:  paymentService.refund(ctx.chargeId))
  .step("reserve-inventory",
        forward:    inventoryService.reserve(order),
        compensate:  inventoryService.release(ctx.reservationId))
  .step("notify-shipping",
        forward:    shippingService.schedule(order),
        compensate:  shippingService.cancel(ctx.shipmentId))
  .build()
  .execute()
```

ON FAILURE AT STEP 2 (reserve-inventory):

Step 1 compensate runs: paymentService.refund(ctx.chargeId)

Saga ends: FAILED with clean state, no dangling charges

WHAT IF COMPENSATION ITSELF FAILS?

Mark saga as COMPENSATION_FAILED in the database.

Alert ops team immediately – do NOT swallow silently.

Manual remediation required (human calls bank, etc.)

SAGA STATE PERSISTED TO DATABASE:

saga_id, request_id, current_step, status

context (JSONB) – stores results of each completed step

failure_step, started_at, completed_at

If pod dies mid-saga: on restart, load saga state from DB,
determine phase (forward or compensation), resume from there.

Me: "The key insight is: in distributed systems, you don't get ACID transactions across service boundaries. You get eventual consistency through compensation. Every step that can succeed must have a compensating action that can undo it. Design your steps to be idempotent — calling refundCard() twice should be safe."

11.4 Tenant Onboarding & GDPR Offboarding

Interviewer: "How do you provision a new tenant? And what happens when a customer cancels and invokes GDPR 'right to erasure'?"

Me: "Onboarding is the first impression. If it takes 3 days to set up a new tenant, you've already lost the customer. I design it to be fully automated and complete in under 60 seconds."

TENANT ONBOARDING PIPELINE

Customer signs up → 60 seconds later they're fully operational

```
POST /admin/tenants
{
  "company": "Acme Corp",
  "tier": "PREMIUM",
  "admin_email": "admin@acme.com"
}
```

```
Step 1 [0-5s]:  Generate tenant_id, API keys, webhook secret
Step 2 [5-10s]: INSERT INTO tenants (tenant_id, config, limits)
Step 3 [10-20s]: Run Flyway baseline migration for new tenant
                  (if schema-per-tenant mode)
Step 4 [20-25s]: Provision rate limit config in Redis
Step 5 [25-30s]: Create Kafka consumer group for tenant
Step 6 [30-40s]: Send welcome email with API credentials
Step 7 [40-60s]: Smoke test: submit test request, verify 202
```

Response:

```
{
  "tenant_id": "acme-corp",
  "api_key": "sk_live_xxxx",
  "webhook_secret": "whsec_xxxx",
  "base_url": "https://api.platform.com/v1",
  "status": "ACTIVE"
}
```

Me: "Now GDPR offboarding is harder. 'Right to erasure' means the customer has the legal right to request complete deletion of all their data within 30 days. Here's what makes it complicated — you have data in multiple places:"

GDPR ERASURE – Where Does Tenant Data Live?

Immediate deletion (within 24 hours):

- ✓ requests table (DELETE WHERE tenant_id=?)
- ✓ idempotency_keys table
- ✓ webhook_events table
- ✓ Redis cache (SCAN + DEL tenant*:acme-corp:*)
- ✓ Kafka consumer group offset (reset + delete)

- Delayed deletion (within 30 days – GDPR deadline):
- ✓ S3 payloads (lifecycle policy on tenant prefix)
 - ✓ CloudWatch logs (log group per tenant, delete group)
 - ✓ X-Ray traces (30-day retention, auto-expires)
 - ✓ RDS backups (delete backups older than 0 days for tenant)

- Legal hold exception (can't delete):
- ✗ Financial transaction records (7 years tax law)
 - Anonymize: replace PII with "GDPR-ERASED-{hash}"
 - Retain aggregates (totals) but not personal details

ERASURE JOB:

@Component ErasureSaga implements stages above
Runs async, sends confirmation email when complete
Stores erasure certificate in immutable audit store

"The key tension: GDPR says delete everything, but financial regulations say keep transaction records for 7 years. The resolution is **anonymization** — you keep the record that 'a payment of \$100 was processed' but you replace all personally-identifiable fields with a pseudonymous token."

11.5 Data Residency & Geo-Routing (GDPR Article 46)

Interviewer: "You have EU customers. Under GDPR, their personal data cannot leave the EU. How does your multi-tenant architecture handle that?"

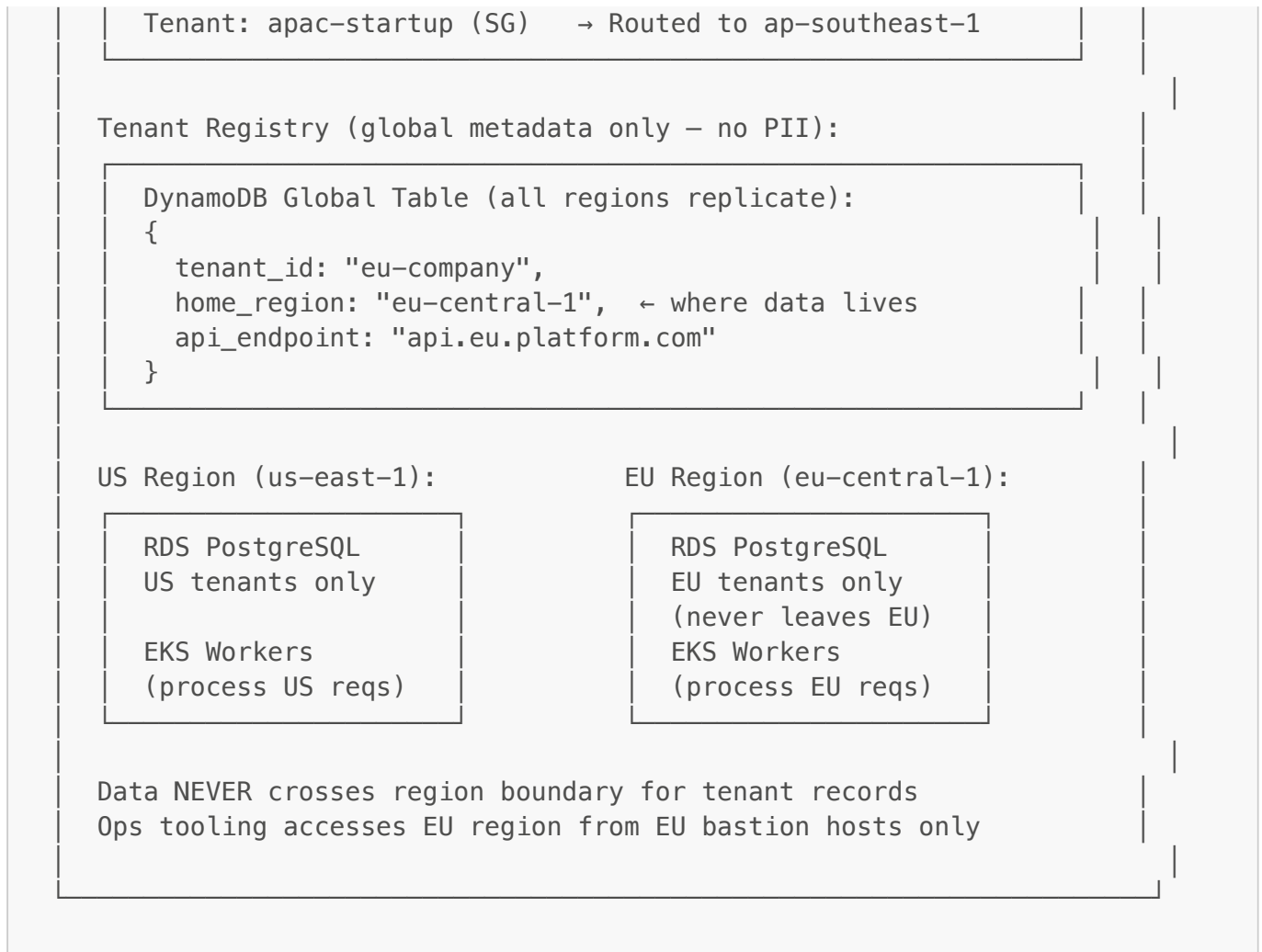
Me: "This is a hard constraint that breaks the 'one shared database' assumption. The moment a German customer's data touches a US server, you have a GDPR violation — regardless of encryption. I've seen companies face €20 million fines for this. Here's how to design for it from day one:"

MULTI-REGION DATA RESIDENCY ARCHITECTURE

Global (DNS + Routing):

Route 53 + CloudFront with Geo-restriction

Tenant: acme-corp (US)	→ Routed to us-east-1
Tenant: eu-company (DE)	→ Routed to eu-central-1



Me: "In practice this means:

- Each tenant has a **home_region** stored in the global registry
- Every API call first resolves which region owns this tenant, then routes there
- EU region services are operated by EU-based engineers with EU access policies in IAM
- Audit logs for EU tenants never leave EU-based CloudWatch

This does add latency for a US engineer debugging an EU tenant's issue — you can't just SSH to **us-east-1** and query the data. That's intentional. Compliance over convenience."

11.6 API Versioning Strategy

Interviewer: "Tenant A is still on v1 of your API. Tenant B just upgraded to v2 which has breaking changes. How do you manage this?"

Me: "API versioning is one of those topics where there's a right answer and five wrong answers. Let me walk you through how I think about it.

First: what constitutes a breaking change? Adding a field is not breaking. Removing a field is. Changing a field's type is. Changing behavior of an existing endpoint is. I document this distinction clearly in our API contract spec."

API VERSIONING STRATEGY

URL-based versioning (what I use):

```
/v1/requests → v1 behavior (maintained)
/v2/requests → v2 behavior (new features)
/v3/requests → v3 in beta
```

Why URL and not header?

→ Headers can't be bookmarked, logged easily, or shared as links
→ URL versioning is explicit, cacheable, debuggable

TENANT VERSION MANAGEMENT:

tenants table:

```
api_version_pinned  VARCHAR(10)  DEFAULT 'v1'
api_version_latest  VARCHAR(10)  DEFAULT 'v2'
version_upgraded_at  TIMESTAMPTZ
```

Tenant A: pinned=v1 (opted out of v2 breaking changes)

Tenant B: pinned=v2 (migrated successfully)

New tenants: pinned=v2 (always start on latest)

SUNSET PROCESS:

```
Month 1: v2 released, v1 stable
Month 3: Deprecation headers on all v1 responses:
         Deprecation: Sat, 01 Feb 2026 00:00:00 GMT
         Sunset: Mon, 01 Jun 2026 00:00:00 GMT
Month 5: Email all v1 tenants with migration guide
Month 8: v1 returns HTTP 410 Gone with migration link
```

SPRING BOOT IMPLEMENTATION:

```
@RequestMapping("/v1") and @RequestMapping("/v2")
on separate controller classes
Shared service layer – controllers handle translation
```

Me: "The critical discipline is: the shared service layer contains the business logic, and versioned controllers handle the translation between old and new API contracts. This means I can maintain v1 and v2 with minimal code duplication — the controller shims the old shape into the new service call. When v1 is finally sunset, I delete the v1 controller, nothing else changes."

11.7 Bulkhead Pattern — Thread Pool Isolation Per Tenant

Interviewer: "If Tenant A has a slow downstream dependency that's causing their requests to take 30 seconds, how do you prevent that from starving threads for Tenant B?"

Me: "This is exactly why you should never use a single shared thread pool in a multi-tenant system. The Bulkhead pattern — borrowed from naval engineering — means you partition resources so a flood in one compartment doesn't sink the ship."

BULKHEAD PATTERN – Thread Pool Per Tenant Tier

WITHOUT BULKHEAD (dangerous):

Shared Thread Pool: 50 threads

Tenant A: 48 threads stuck waiting on slow API →
Tenant B: needs a thread → STARVED

WITH BULKHEAD (safe):

Tenant A thread pool: 20 threads (max)
Tenant A: 20 threads stuck → Tenant A's requests queue up
Tenant A gets 429 on 21st request

Tenant B thread pool: 20 threads (max)
Tenant B: completely unaffected by Tenant A's issues

Default thread pool: 10 threads (shared, low priority)

RESILIENCE4J IMPLEMENTATION:

```
// In application.yml:
resilience4j.bulkhead:
  instances:
    tenant-a:
      maxConcurrentCalls: 20
      maxWaitDuration: 100ms
    tenant-b:
      maxConcurrentCalls: 20
      maxWaitDuration: 100ms

// In code:
@Bulkhead(name = "{tenantId}", fallbackMethod = "rateLimitFallback")
public Result processForTenant(String tenantId, Request req) {
    return workerService.execute(req);
}
```

"Combined with the circuit breaker pattern — where after 5 consecutive failures for a tenant's downstream, we open the circuit and fail-fast for 30 seconds — this gives complete blast radius containment. Tenant A's infrastructure issues stay tenant A's problem."

11.8 Principal-Level Closing Statement

Me: "Here's what separates a 15-year perspective from a 10-year perspective on multi-tenant design. At 10 years, you know the patterns. You can build the async flow, the outbox, the retry logic. You've shipped these things.

At 15 years, you've been woken up at 2 AM by the things you didn't design for. You've seen the GDPR letter. You've watched a noisy tenant bring down production. You've done the emergency schema migration at midnight. And you've learned that the hard questions aren't 'how do I build this' — they're:

- How do I isolate tenants when the blast radius matters?
- How do I change the schema without downtime on 100M rows?
- How do I handle multi-step workflows where any step can fail?
- How do I delete a customer's data everywhere it lives?
- How do I evolve the API without breaking existing integrations?

The architecture I described handles all of these. Not because I over-engineered it — but because each piece exists to solve a failure I've seen in production. The outbox pattern is there because I've seen Kafka go down. Row-level security is there because I've seen a missing WHERE clause leak data. The expand-contract pattern is there because I've seen a 4 AM ALTER TABLE take the whole system down.

Production experience isn't about knowing more patterns. It's about knowing which problems each pattern was invented to solve."

11.9 Row-Level Security (PostgreSQL RLS) — DB-Enforced Tenant Isolation

Me: "One of the most dangerous failure modes in a multi-tenant system is a missing `WHERE tenant_id = ?` clause. A developer writes a query, ships it, and suddenly Tenant A can see Tenant B's data. The application layer filters data by tenant — but the DB has no idea. One bug and you have a data breach.

PostgreSQL Row-Level Security moves the isolation enforcement into the database itself. Even if the application forgets the tenant filter, the DB silently applies it. This is defence in depth for multi-tenancy."

The Problem: Application-Level Filtering is Fragile

Application-level filtering:

```
SELECT * FROM orders WHERE tenant_id = 'tenant-abc';
```

↑

Developer MUST remember this.
One missing WHERE clause = data leak.
No DB-level safety net.


```
// Easy to forget in a complex JOIN:
@Query("SELECT o FROM Order o JOIN o.items i WHERE i.productId =
:productId")
List<Order> findByProduct(@Param("productId") Long productId);
// ❌ Missing tenant_id filter – returns all tenants' orders
```

PostgreSQL RLS: The DB Enforces Isolation

```
-- Step 1: Enable RLS on the table
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE orders FORCE ROW LEVEL SECURITY; -- applies to table owner too

-- Step 2: Create policy – DB applies this WHERE clause to EVERY query
CREATE POLICY tenant_isolation_policy ON orders
    USING (tenant_id = current_setting('app.current_tenant_id')::UUID);

-- Now even this dangerous query is safe:
SELECT * FROM orders;
-- DB automatically rewrites to:
-- SELECT * FROM orders WHERE tenant_id =
-- current_setting('app.current_tenant_id')::UUID
-- Tenant A CANNOT see Tenant B's rows – ever.
```

Spring Boot Integration: Setting the Tenant Context in the DB Session

```
// Approach 1: Hibernate interceptor – sets tenant per transaction
@Component
public class TenantAwareHibernateInterceptor extends EmptyInterceptor {

    @Override
    public void beforeTransactionCompletion(Transaction tx) {
        // no-op
    }
}

// Approach 2 (Recommended): JPA EntityManager with session variable
@Repository
public class TenantAwareOrderRepository {

    @PersistenceContext
    private EntityManager entityManager;

    private void setTenantContext(String tenantId) {
        // Sets PostgreSQL session variable before every query
        entityManager.createNativeQuery(
```

```

        "SELECT set_config('app.current_tenant_id', :tenantId, true)"
    ).setParameter("tenantId", tenantId)
    .getSingleResult();
}

public List<Order> findAllOrders() {
    String tenantId = TenantContext.getCurrentTenant(); // from
ThreadLocal
    setTenantContext(tenantId);
    // RLS policy now active – DB filters automatically
    return entityManager.createQuery("SELECT o FROM Order o",
Order.class)
        .getResultList();
}
}

```

Approach 3: Spring AOP — Automatically Set Tenant on Every Repository Call

```

// Cleaner: AOP aspect sets tenant context before every DB call
@Aspect
@Component
public class RlsTenantAspect {

    @PersistenceContext
    private EntityManager entityManager;

    @Before("execution(* com.example.repository..*(..))")
    public void setTenantContext() {
        String tenantId = TenantContext.getCurrentTenant();
        if (tenantId != null) {
            entityManager.createNativeQuery(
                "SELECT set_config('app.current_tenant_id', :tenantId,
true)"
            ).setParameter("tenantId", tenantId)
            .getSingleResult();
        }
    }
}

```

Approach 4: HikariCP Connection Initialisation (Most Performant)

```

@Configuration
public class DataSourceConfig {

    @Bean
    public DataSource tenantAwareDataSource() {
        HikariConfig config = new HikariConfig();
        config.setJdbcUrl("jdbc:postgresql://localhost:5432/saasdb");
    }
}

```

```

        config.setUsername("app_user");
        config.setPassword("secret");

        // Set tenant context on every connection checkout from pool
        config.setConnectionInitSql(
            "SELECT set_config('app.current_tenant_id', '', false)"
        );
        return new HikariDataSource(config);
    }
}

// Then set per-request via interceptor:
@Component
public class TenantInterceptor implements HandlerInterceptor {

    @Autowired
    private DataSource dataSource;

    @Override
    public boolean preHandle(HttpServletRequest request,
                             HttpServletResponse response, Object handler)
        throws Exception {
        String tenantId = extractTenantFromJwt(request);
        TenantContext.setCurrentTenant(tenantId);

        // Set in DB session for this request's connection
        try (Connection conn = dataSource.getConnection()) {
            conn.createStatement().execute(
                "SELECT set_config('app.current_tenant_id', '" + tenantId +
                "', true)"
            );
        }
        return true;
    }
}

```

Multiple Policies: Read vs Write Isolation

```

-- Separate policies for SELECT and INSERT/UPDATE/DELETE
-- Prevents accidental cross-tenant writes too

-- Read policy
CREATE POLICY orders_read_policy ON orders
    FOR SELECT
    USING (tenant_id = current_setting('app.current_tenant_id')::UUID);

-- Write policy – tenant can only insert rows for themselves
CREATE POLICY orders_write_policy ON orders
    FOR INSERT
    WITH CHECK (tenant_id =

```

```
current_setting('app.current_tenant_id')::UUID);

-- Update/Delete policy
CREATE POLICY orders_modify_policy ON orders
  FOR ALL
  USING (tenant_id = current_setting('app.current_tenant_id')::UUID)
  WITH CHECK (tenant_id =
current_setting('app.current_tenant_id')::UUID);

-- Platform admin bypass (superuser role bypasses RLS)
-- Create a separate DB role for platform admin queries:
CREATE ROLE platform_admin BYPASSRLS;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO platform_admin;
```

RLS vs Application-Level Filtering — When to Use Which

	Application Filtering	PostgreSQL RLS
Safety	Only as good as code	DB enforces always
Bug risk	One missing WHERE = data breach	DB silently filters even on buggy queries
Performance	No overhead	Tiny overhead (index on tenant_id)
Complex queries	Developer handles	Automatic
Platform admin view	Easy (no filter)	BYPASSRLS role
Cross-tenant reports	Easy	Needs BYPASSRLS role
ORM compatibility	Fully compatible	Needs session var setup
Audit / compliance	Trust the developer	DB-enforced – auditable

Recommendation:
Use BOTH – defence in depth.
Application filter: primary, explicit, readable in code.
RLS policy: safety net, catches any missed filter.

RLS with AWS RDS PostgreSQL

```
-- RDS supports RLS fully (PostgreSQL 9.5+)
-- Use parameter groups to ensure pg_stat_statements is on for auditing

-- Connection string for Spring Boot with RDS:
spring:
  datasource:
    url: jdbc:postgresql://your-rds-endpoint.rds.amazonaws.com:5432/saasdb
    username: app_user -- NOT superuser – superuser bypasses RLS
    password: ${DB_PASSWORD} -- from AWS Secrets Manager
```

```
-- IMPORTANT: app_user must NOT have SUPERUSER or BYPASSRLS
-- Only the platform_admin role should bypass RLS
```

Interview Talking Points

Q: Why not just use application-level filtering? Application filters are only as reliable as the developer writing the query. One missed `WHERE tenant_id = ?` in a JOIN, a native SQL query, or a reporting query causes a data breach. RLS is a database-enforced safety net — it cannot be bypassed by application bugs.

Q: How do you handle platform admin queries that need all tenants? Create a separate DB role `platform_admin` `BYPASSRLS`. Application service uses `app_user` (RLS active). Admin tools/reporting use `platform_admin` (bypasses RLS). Never give the application user `BYPASSRLS`.

Q: Does RLS work with JPA/Hibernate? Yes — you set the PostgreSQL session variable `app.current_tenant_id` before each transaction via interceptor, AOP aspect, or HikariCP connection hook. Hibernate queries are filtered transparently.

Q: Performance impact of RLS? Negligible with proper indexing. Always index `tenant_id` on every RLS-protected table. PostgreSQL uses the index for the RLS filter just like any other WHERE clause.

Q: How does this compare to schema-per-tenant isolation? RLS is simpler operationally (one schema, one connection pool) but requires trusting the DB policy. Schema-per-tenant gives stronger isolation but has connection pool limits and migration complexity. For most SaaS: RLS + shared schema. For enterprise/regulated: schema or DB per tenant.

END OF DOCUMENT